

Guia de estudos

Preparação para o workshop Attention is All You Need

Giovanni Vasconcelos · CESUPA Tech Summit 2026

Sumário

Como usar este guia	1
Cronograma sugerido (três semanas)	2
Módulo 0. Revisão de pré-requisitos	2
Módulo 1. Leitura guiada do artigo	2
Módulo 2. Atenção em profundidade	3
Módulo 3. Os demais componentes	5
Módulo 4. Treino	6
Módulo 5. Geração	6
Módulo 6. Do GPT aos grandes modelos de linguagem	7
Módulo 7. Xadrez e modelos de mundo	8
Módulo 8. Domínio do código do workshop	8
Módulo 9. Perguntas difíceis da plateia	9
Lista de autoverificação	10
Referências	10

Como usar este guia

O guia supõe familiaridade com aprendizado de máquina e com a ideia geral de um Transformer. O objetivo é outro: chegar ao workshop capaz de derivar cada peça no quadro, justificar cada escolha de projeto e responder perguntas fora do roteiro sem hesitar.

Cada módulo tem três partes: o que estudar (com fontes), os pontos que você deve conseguir explicar e exercícios de verificação. Os exercícios são a parte mais importante. Se você não consegue resolver um deles sem consultar material, o módulo não está pronto.

Cronograma sugerido (três semanas)

Semana	Módulos	Carga aproximada
1	0 (revisão), 1 (artigo), 2 (atenção)	8 a 10 h
2	3 (componentes), 4 (treino), 5 (geração), 8 (código do workshop)	8 a 10 h
3	6 (do GPT aos LLMs), 7 (xadrez e modelos de mundo), 9 (perguntas difíceis), ensaio	6 a 8 h

Uma ordem de leitura que funciona bem: primeiro o vídeo de Karpathy (Módulo 8), que dá a visão de conjunto em código; depois o artigo, lido com o roteiro do Módulo 1; e só então os aprofundamentos.

Módulo 0. Revisão de pré-requisitos

Estudar.

- Álgebra linear: produto escalar como medida de similaridade, multiplicação de matrizes em termos de linhas e colunas, projeções lineares. Referência: Prince (2023), apêndice B; ou o capítulo 2 de Goodfellow, Bengio e Courville (2016).
- Probabilidade: softmax, entropia cruzada, relação entre entropia cruzada e máxima verossimilhança. Goodfellow et al. (2016), seções 3.13 e 5.5.
- Retropropagação e diferenciação automática. Goodfellow et al. (2016), seção 6.5.
- PyTorch: tensores, broadcasting, `nn.Module`, `register_buffer`, `torch.no_grad`, o ciclo `zero_grad/backward/step`.

Você deve conseguir explicar.

- Por que a entropia cruzada é a perda natural para classificação sobre um vocabulário.
- A diferença entre `x.T` e `x.transpose(-2, -1)` em um tensor de três dimensões.
- O que `keepdims=True` muda no broadcasting.

Exercícios.

1. Mostre que $\text{softmax}(z) = \text{softmax}(z - c)$ para qualquer escalar c .
2. Mostre que o jacobiano da softmax é $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$, isto é, $J = \text{diag}(p) - pp^T$. Conclua que, quando p se aproxima de um vetor one-hot, todas as entradas de J tendem a zero.
3. Um modelo que prevê a distribuição uniforme sobre 33 símbolos tem perda $\ln 33 \approx 3,50$. Confira esse valor e relacione com a perda inicial impressa no treino de xadrez.

Módulo 1. Leitura guiada do artigo

Estudar. Vaswani et al. (2017), *Attention Is All You Need*. Leia o artigo inteiro duas vezes: uma leitura corrida e uma segunda com as perguntas abaixo. A nota 2017-01-01_vaswani-attention-is-all-you-need do seu vault tem um resumo útil para revisão.

Roteiro por seção.

- **Seções 1 e 2 (introdução e contexto).** Qual é o problema das RNNs que o artigo ataca? Qual era o papel da atenção antes de 2017 (Bahdanau et al., 2015)? O que exatamente o título quer dizer com “all you need”?
- **Seção 3.1 (arquitetura).** Desenhe a Figura 1 de memória. Identifique as três formas de atenção: self-attention no encoder, self-attention mascarada no decoder e atenção encoder-decoder.
- **Seção 3.2 (atenção).** Leia a nota de rodapé 4, que justifica o fator $1/\sqrt{d_k}$. Compare atenção aditiva e multiplicativa: por que os autores preferem a multiplicativa?
- **Seção 3.3 (FFN).** Por que $d_{ff} = 4 \cdot d_{model}$? (Não há justificativa teórica forte; é uma escolha empírica que se manteve.)
- **Seção 3.4 (embeddings).** Os pesos da camada de embedding e da camada linear final são compartilhados, e os embeddings são multiplicados por $\sqrt{d_{model}}$. Por quê?
- **Seção 3.5 (codificação de posição).** Por que senos e cossenos? Qual o argumento sobre posições relativas?
- **Seção 4 (por que self-attention).** Explique cada coluna da Tabela 1. Esta tabela é o slide 7 do workshop.
- **Seção 5 (treino).** Anote os hiperparâmetros: Adam com $\beta_1 = 0,9$, $\beta_2 = 0,98$, $\epsilon = 10^{-9}$; *warmup* de 4000 passos; dropout 0,1; *label smoothing* 0,1. O modelo base treinou 100 mil passos em 12 horas em 8 GPUs P100; o grande, 300 mil passos em 3,5 dias.
- **Seção 6 (resultados).** 28,4 BLEU em inglês-alemão e 41,8 em inglês-francês (WMT 2014). Leia a Tabela 3 com atenção: ela é um estudo de ablação e responde a várias perguntas prováveis da plateia (efeito do número de cabeças, de d_k , de dropout, de embeddings de posição aprendidos).

Você deve conseguir explicar.

- O fluxo de dados completo do modelo de tradução, da frase em inglês à distribuição sobre a próxima palavra em alemão.
- A diferença entre as três formas de atenção e de onde vêm Q, K e V em cada uma.
- A taxa de aprendizado do artigo: $lr = d_{model}^{-0,5} \cdot \min(\text{step}^{-0,5}, \text{step} \cdot \text{warmup}^{-1,5})$. Esboce a curva.

Exercícios.

1. Conte os parâmetros do modelo base ($N = 6$, $d_{model} = 512$, $d_{ff} = 2048$, vocabulário compartilhado de cerca de 37 mil tokens) e confirme que o total fica próximo de 65 milhões.
2. Segundo a Tabela 3, o que acontece com o BLEU quando se usa uma única cabeça? E com 32 cabeças?

Módulo 2. Atenção em profundidade

Estudar.

- Prince (2023), capítulo 12, seções 12.1 a 12.4.

- Alammr (2018), *The Illustrated Transformer*.
- 3Blue1Brown (2024), capítulos 5 e 6 da série sobre redes neurais (*Transformers e Attention in transformers, visually explained*). Excelente fonte de analogias visuais para a exposição.
- Bahdanau, Cho e Bengio (2015), para o contexto histórico.

Pontos centrais.

Atenção como média ponderada. A saída na posição i é $\sum_j \alpha_{ij} v_j$, com $\alpha_{ij} = \text{softmax}_j(q_i \cdot k_j / \sqrt{d_k})$. É um estimador de Nadaraya–Watson com núcleo exponencial: uma regressão por núcleo em que a similaridade é aprendida. Essa leitura ajuda a explicar a analogia do dicionário do slide 8.

O fator de escala. Se q_i, k_i são independentes, com média 0 e variância 1, então $\text{Var}(q_i k_i) = 1$ e $\text{Var}(q \cdot k) = d_k$. O desvio-padrão dos scores cresce como $\sqrt{d_k}$; a softmax satura e, pelo Exercício 2 do Módulo 0, os gradientes desaparecem. Dividir por $\sqrt{d_k}$ restaura a variância unitária. O experimento da seção 1.3 do notebook mede isso: com 10 keys e $d_k = 512$, o maior peso médio sem escala é 0,95, e com escala, 0,31.

A máscara causal. Somar $-\infty$ antes da softmax zera exatamente os pesos proibidos e renormaliza os restantes. A consequência importante é o paralelismo no treino: uma passada pela sequência produz T previsões ao mesmo tempo, todas com o contexto correto. Isso é chamado de *teacher forcing*, e é a razão principal de o treino de Transformers ser tão mais eficiente que o de RNNs.

Por que várias cabeças. Uma única softmax por posição produz uma única média ponderada. Com h cabeças, a mesma posição pode agregar informação de lugares diferentes, por critérios diferentes. Como cada cabeça tem dimensão d_{model}/h , o custo total é praticamente o mesmo de uma cabeça de dimensão cheia. Existe um argumento de posto: QK^T tem posto no máximo d_k , então cada cabeça é um mapa de atenção de posto baixo.

Complexidade. Tempo $O(T^2 d)$ e memória $O(T^2)$ por cabeça, por causa da matriz de scores. É isso que limita o tamanho do contexto. FlashAttention (Dao et al., 2022) calcula a atenção exata sem materializar a matriz $T \times T$ na memória da GPU, reorganizando o cálculo em blocos.

Invariância a permutações. Sem codificação de posição, permutar a entrada apenas permuta a saída (a atenção é equivariante a permutações). Sem a informação de ordem, “o rei protege a rainha” e “a rainha protege o rei” teriam representações idênticas, a menos da ordem das linhas.

Exercícios.

1. Implemente a atenção em NumPy sem consultar o notebook, incluindo a máscara. Compare com `checagens._atencao_ref`.
2. Demonstre formalmente a equivariância a permutações: para uma matriz de permutação P , $\text{Attention}(PX) = P \cdot \text{Attention}(X)$ quando não há máscara nem codificação de posição.
3. O que acontece com os pesos de atenção se todas as keys forem iguais? E se a query for o vetor nulo?
4. Se dividirmos por d_k em vez de $\sqrt{d_k}$, qual o efeito sobre a distribuição dos pesos para d_k grande?

Módulo 3. Os demais componentes

Estudar.

- He et al. (2016), para conexões residuais; Ba, Kiros e Hinton (2016), para LayerNorm.
- Xiong et al. (2020), *On Layer Normalization in the Transformer Architecture*: a comparação entre pre-LN e post-LN.
- Elhage et al. (2021), *A Mathematical Framework for Transformer Circuits*: a leitura do Transformer como um *residual stream* em que cada camada lê e escreve.
- Geva et al. (2021), *Transformer Feed-Forward Layers Are Key-Value Memories*.
- Su et al. (2021), RoPE; Press, Smith e Lewis (2022), ALiBi.

Pontos centrais.

Conexões residuais. Cada subcamada aprende uma correção $x \mapsto x + f(x)$, e o gradiente tem um caminho direto até as camadas iniciais. Na visão de Elhage et al., o vetor de cada posição é um canal de comunicação compartilhado: a atenção move informação entre posições e a FFN processa informação dentro de uma posição.

Pre-LN e post-LN. O artigo usa post-LN, $x = \text{LN}(x + f(x))$. O workshop usa pre-LN, $x = x + f(\text{LN}(x))$, como o GPT-2 e quase todos os modelos atuais. Xiong et al. (2020) mostram que o post-LN produz gradientes grandes perto da saída no início do treino, o que exige *warmup* cuidadoso; o pre-LN treina de forma estável sem isso. Esta é uma pergunta provável de quem tiver lido o artigo.

FFN. Duas camadas lineares com ativação entre elas, aplicadas a cada posição de forma independente. Geva et al. (2021) interpretam a primeira camada como um conjunto de chaves que detectam padrões e a segunda como os valores correspondentes, o que aproxima a FFN de uma memória associativa. A maior parte dos parâmetros de um Transformer está nas FFNs.

Codificação de posição. Três famílias. Absoluta senoidal (o artigo); absoluta aprendida (GPT-2 e o workshop); relativa, em que a posição entra no cálculo da atenção (RoPE, que rotaciona queries e keys conforme a posição e é usada no Llama; ALiBi, que soma uma penalidade proporcional à distância). As relativas generalizam melhor para contextos maiores que os vistos no treino.

Contagem de parâmetros. Por bloco, ignorando vieses e LayerNorm: $4d^2$ na atenção (W_Q, W_K, W_V, W_O) e $8d^2$ na FFN (duas matrizes $d \times 4d$), total $12d^2$. O modelo da turma ($d = 128$, 4 blocos) tem $12 \cdot 128^2 \cdot 4 \approx 786$ mil parâmetros nos blocos, mais cerca de 40 mil nos embeddings: 0,83 milhão, como o notebook imprime. O de referência ($d = 256$, 6 blocos) tem cerca de 4,8 milhões.

Exercícios.

1. Refaça a contagem de parâmetros do modelo da turma incluindo vieses e LayerNorms e compare com `modelo.n_params()`.
2. Implemente a codificação senoidal como alternativa ao `pos_emb` (é o primeiro exercício para casa do notebook) e compare as curvas de perda.
3. Explique por que, com codificação senoidal, PE_{pos+k} é uma função linear de PE_{pos} .

Módulo 4. Treino

Estudar.

- Loshchilov e Hutter (2019), AdamW.
- Szegedy et al. (2016), *label smoothing*.
- Micikevicius et al. (2018), treino com precisão mista.

Pontos centrais.

- O objetivo é a entropia cruzada média do próximo token, $\mathcal{L} = -\frac{1}{T} \sum_t \log p_\theta(x_{t+1} \mid x_{\leq t})$. Minimizá-la equivale a maximizar a verossimilhança dos dados.
- AdamW separa o *weight decay* da atualização adaptativa do Adam, o que regulariza de forma mais previsível.
- *Warmup* seguido de decaimento (no workshop, cosseno no `pretrainar.py`; no artigo, inverso da raiz quadrada). O *warmup* evita atualizações grandes enquanto as estimativas de segundo momento do Adam ainda são ruins.
- Precisão mista: as multiplicações de matrizes rodam em fp16 e os pesos mestres ficam em fp32. O `GradScaler` multiplica a perda por um fator grande antes do *backward*, para que gradientes pequenos não virem zero em fp16.
- *Gradient clipping* (usado no pré-treino) limita a norma do gradiente e evita passos catastróficos.

Exercícios.

1. No notebook, a tarefa de ordenação usa alvo -1 nas posições da entrada. Explique por que, e o que aconteceria se essas posições entrassem na perda.
2. Por que a perda de validação da Parte 3 fica próxima da perda de treino? O que você esperaria ver se o modelo fosse muito maior e treinado por muito mais tempo?

Módulo 5. Geração

Estudar.

- Holtzman et al. (2020), *The Curious Case of Neural Text Degeneration* (amostragem por núcleo, ou *top-p*).
- Pope et al. (2023), sobre inferência eficiente e KV cache (leitura opcional; o conceito basta).

Pontos centrais.

- Geração autorregressiva: amostra-se um token, ele é anexado ao contexto, e o modelo é executado de novo.
- Temperatura τ : divide os logits antes da softmax. $\tau \rightarrow 0$ aproxima a escolha gulosa; τ grande aproxima a distribuição uniforme.
- *Top-k* e *top-p*: truncam a distribuição antes de amostrar, eliminando a cauda de tokens improváveis. No xadrez, isso reduziria lances ilegais.

- KV cache: durante a geração, as keys e values das posições anteriores não mudam, então podem ser guardadas. Cada novo token custa $O(T \cdot d)$ em vez de recalculá-lo tudo. A implementação do workshop não usa cache (por simplicidade), e por isso recalcula a sequência inteira a cada token.

Exercícios.

1. Implemente *top-k* no método *generate* e compare a taxa de legalidade do modelo da turma com $k = 3$.
2. Calcule quantas operações a geração de 100 tokens custa com e sem KV cache, em função de T .

Módulo 6. Do GPT aos grandes modelos de linguagem

Estudar.

- Radford et al. (2018, 2019), GPT e GPT-2; Brown et al. (2020), GPT-3 e o aprendizado em contexto.
- Devlin et al. (2019), BERT, para o contraste com os modelos encoder-only.
- Sennrich, Haddow e Birch (2016), tokenização por BPE.
- Kaplan et al. (2020) e Hoffmann et al. (2022), leis de escala. A conclusão de Hoffmann et al. (o modelo “Chinchilla”) é que, para um orçamento fixo de computação, o número de tokens de treino deve crescer na mesma proporção que o número de parâmetros, em torno de 20 tokens por parâmetro.
- Ouyang et al. (2022), InstructGPT: ajuste por instruções e aprendizado por reforço com feedback humano (RLHF).

Pontos centrais.

- O modelo decoder-only venceu porque o objetivo “prever o próximo token” transforma qualquer texto em dado de treino, sem rotulação, e escala de forma previsível.
- Um modelo pré-treinado é um completador de texto. O comportamento de assistente vem de etapas posteriores: ajuste supervisionado com exemplos de instruções e respostas, seguido de otimização por preferências humanas ou por recompensas verificáveis.
- Diferenças arquiteturais comuns hoje em relação ao workshop: RMSNorm no lugar de LayerNorm, ativação SwiGLU na FFN, RoPE, *grouped-query attention* (várias cabeças de query compartilham keys e values, o que reduz o KV cache) e, em alguns modelos, *mixture of experts* na FFN.

Exercício. Monte uma tabela com três linhas (GPT-2, GPT-3, um modelo aberto recente da família Llama) e colunas para número de parâmetros, número de camadas, d_{model} , número de cabeças, tamanho do contexto e tokens de treino. Use os artigos ou os *model cards* originais.

Módulo 7. Xadrez e modelos de mundo

Estudar.

- Karvonen (2024), *Emergent World Models and Latent Variable Estimation in Chess-Playing Language Models*, arXiv:2403.15498. É a referência direta da Parte 3: o formato de dados do workshop é o dele.
- Li et al. (2023), *Emergent World Representations*: o experimento Othello-GPT.
- Nanda, Lee e Wattenberg (2023), *Emergent Linear Representations in World Models of Self-Supervised Sequence Models*, que mostra que a representação do tabuleiro no Othello-GPT é linear quando expressa como “minhas peças” e “peças do adversário”.
- Alain e Bengio (2016), sobre sondas lineares (*linear probes*) como método.

Pontos centrais.

- Uma sonda linear é um classificador linear treinado sobre as ativações congeladas do modelo para prever uma propriedade (por exemplo, “que peça está em e4”). Se uma sonda linear acerta, a informação está presente e é linearmente acessível naquela camada.
- Karvonen mostra que modelos treinados apenas com PGN desenvolvem representações do tabuleiro e até de uma variável latente de habilidade do jogador, e que intervir nessas representações altera o comportamento do modelo.
- A taxa de legalidade do workshop é uma métrica comportamental, mais fraca que uma sonda: ela mostra que o modelo age como se soubesse a posição, sem mostrar onde nem como essa informação está representada.
- Limitações a reconhecer: o modelo da turma é pequeno e treinado por minutos. Ele acerta aberturas porque elas são frequentes nos dados, e degrada no meio-jogo, onde as posições são novas. Parte do que parece compreensão é memorização de sequências comuns.

Exercícios.

1. Esboce como você treinaria uma sonda linear para a casa e4 usando o modelo de referência: que ativações coletar, qual o alvo, como separar treino e teste.
2. Explique por que o formato de caracteres dificulta a sonda (um lance ocupa vários tokens) e como Karvonen lida com isso (as sondas são aplicadas na posição do ponto que precede cada lance das brancas).

Módulo 8. Domínio do código do workshop

Estudar.

- Karpathy (2023), *Let's build GPT: from scratch, in code, spelled out* (vídeo, cerca de 2 horas) e o repositório nanoGPT. O código do workshop segue o mesmo estilo, com nomes em português.
- Os arquivos do repositório: `mini_gpt.py`, `checagens.py`, `scripts/gerar_notebooks.py`, `scripts/pretreinar.py`, `scripts/preparar_dados.py`.

Você deve conseguir, sem consulta.

- Escrever Head, MultiHeadAttention e Block do zero.
- Explicar cada linha de GPT.forward com o shape de cada tensor.
- Explicar como DadosXadrez.batch monta os exemplos (toda janela começa em um ;, isto é, no início de uma partida) e por que isso importa para o modelo aprender a notação dos números de lance.
- Explicar como lance_do_modelo monta o prompt (histórico em PGN terminado em N. ou em espaço, conforme a vez) e o que acontece quando o modelo erra.
- Explicar a diferença entre o modelo da turma (4 blocos, 4 cabeças, $d = 128$, contexto 256) e o de referência (6 blocos, 8 cabeças, $d = 256$, contexto 384).

Exercícios.

1. Resolva o workshop_aluno.ipynb inteiro sem consultar o gabarito e cronometre.
2. Introduza de propósito cada erro da tabela da seção 5 do plano e observe a mensagem das verificações. Assim você reconhece o sintoma na sala.
3. Execute o notebook de pré-treino e anote a taxa de legalidade do modelo de referência.

Módulo 9. Perguntas difíceis da plateia

Respostas curtas, para ter prontas.

“Por que se chama atenção?” O nome vem da tradução automática neural (Bahdanau et al., 2015): ao gerar cada palavra, o modelo “presta atenção” a partes diferentes da frase de origem. Matematicamente, é uma média ponderada com pesos que dependem dos dados.

“Por que o custo é quadrático e isso importa?” Cada posição se compara com todas as outras: T^2 comparações por cabeça e por camada. Para contextos de centenas de milhares de tokens, isso domina o custo; daí FlashAttention, atenção esparsa e alternativas como modelos de espaço de estados.

“O modelo entende xadrez?” Depende do que se entende por entender. Há evidência de que modelos desse tipo representam internamente o estado do tabuleiro (Karvonen, 2024). Por outro lado, o modelo da turma não planeja, não calcula variantes e erra regras no meio-jogo. É uma boa oportunidade para distinguir comportamento, representação e compreensão.

“Qual a diferença para o AlphaZero ou o Stockfish?” O AlphaZero recebe o tabuleiro como entrada, combina uma rede com busca em árvore (MCTS) e aprende por reforço jogando contra si mesmo. O Stockfish usa busca alfa-beta com uma função de avaliação. O modelo do workshop não faz busca nenhuma e nunca vê o tabuleiro: apenas imita partidas humanas.

“Por que caracteres e não palavras ou lances inteiros?” Simplicidade: 33 símbolos e nenhum tokenizador para explicar. O custo é que cada lance vira vários tokens e as sequências ficam longas. LLMs usam BPE, que é um meio-termo entre caracteres e palavras.

“Os pesos de atenção explicam o que o modelo faz?” Só parcialmente. Jain e Wallace (2019) mostram que pesos de atenção diferentes podem produzir a mesma saída. Os pesos dizem de onde

a informação é lida, não o que é feito com ela.

“Por que dividir por $\sqrt{d_k}$ e não normalizar de outro jeito?” É a correção mínima que torna a variância dos scores independente da dimensão, supondo componentes independentes com variância unitária. Variantes modernas às vezes normalizam queries e keys com LayerNorm (*QK-norm*), com o mesmo objetivo.

“O que é o KV cache?” Na geração, as keys e values das posições passadas não mudam, então são armazenadas e reutilizadas. É o principal consumidor de memória na inferência de LLMs.

“Como o GPT passou a seguir instruções?” Ajuste supervisionado com pares de instrução e resposta, seguido de aprendizado por reforço com preferências humanas (Ouyang et al., 2022). A arquitetura não muda.

“O que é o contexto de um modelo?” O número máximo de tokens que ele processa de uma vez (`block_size` no workshop). No código, `generate` corta o histórico para os últimos `block_size` tokens; em partidas muito longas, o modelo perde o início da partida.

“Transformers só servem para texto?” Não. Vision Transformers dividem imagens em blocos e tratam cada bloco como um token (Dosovitskiy et al., 2021). O mesmo vale para áudio, proteínas e, como no workshop, partidas de xadrez.

“Por que o encoder não usa máscara?” Porque na tradução a frase de origem inteira está disponível desde o início. A máscara só é necessária quando o modelo gera a sequência e não pode ver o futuro.

Lista de autoverificação

Antes do ensaio (D – 7), você deve conseguir, sem consulta:

- Derivar a variância de $q \cdot k$ e explicar a saturação da softmax com o jacobiano.
- Desenhar a Figura 1 do artigo e explicar as três formas de atenção.
- Explicar a Tabela 1 do artigo coluna por coluna.
- Escrever `Head`, `MultiHeadAttention` e `Block` em PyTorch sem erros de shape.
- Contar os parâmetros do modelo da turma.
- Explicar pre-LN e post-LN e por que o workshop diverge do artigo.
- Explicar por que a máscara causal permite treinar T previsões em paralelo.
- Explicar temperatura, $top-k$, $top-p$ e KV cache.
- Explicar o que é uma sonda linear e o resultado de Karvonen (2024).
- Responder às perguntas do Módulo 9 em menos de um minuto cada.

Referências

- Alain, G.; Bengio, Y. (2016). Understanding intermediate layers using linear classifier probes. arXiv:1610.01644.
- Alamm, J. (2018). The Illustrated Transformer. jalammar.github.io/illustrated-transformer.

- Ba, J. L.; Kiros, J. R.; Hinton, G. E. (2016). Layer Normalization. arXiv:1607.06450.
- Bahdanau, D.; Cho, K.; Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
- Dao, T. et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS.
- Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
- Dosovitskiy, A. et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR.
- Elhage, N. et al. (2021). A Mathematical Framework for Transformer Circuits. Transformer Circuits Thread.
- Geva, M. et al. (2021). Transformer Feed-Forward Layers Are Key-Value Memories. EMNLP.
- Goodfellow, I.; Bengio, Y.; Courville, A. (2016). Deep Learning. MIT Press.
- He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR.
- Hoffmann, J. et al. (2022). Training Compute-Optimal Large Language Models. NeurIPS.
- Holtzman, A. et al. (2020). The Curious Case of Neural Text Degeneration. ICLR.
- Jain, S.; Wallace, B. C. (2019). Attention is not Explanation. NAACL.
- Kaplan, J. et al. (2020). Scaling Laws for Neural Language Models. arXiv:2001.08361.
- Karpathy, A. (2023). Let's build GPT: from scratch, in code, spelled out. Video (YouTube) e repositório nanoGPT.
- Karvonen, A. (2024). Emergent World Models and Latent Variable Estimation in Chess-Playing Language Models. arXiv:2403.15498.
- Li, K. et al. (2023). Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. ICLR.
- Loshchilov, I.; Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR.
- Micikevicius, P. et al. (2018). Mixed Precision Training. ICLR.
- Nanda, N.; Lee, A.; Wattenberg, M. (2023). Emergent Linear Representations in World Models of Self-Supervised Sequence Models. BlackboxNLP.
- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. NeurIPS.
- Pope, R. et al. (2023). Efficiently Scaling Transformer Inference. MLSys.
- Press, O.; Smith, N. A.; Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. ICLR.
- Prince, S. J. D. (2023). Understanding Deep Learning. MIT Press.
- Radford, A. et al. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI.
- Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI.
- Sennrich, R.; Haddow, B.; Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.
- Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.

- Szegedy, C. et al. (2016). Rethinking the Inception Architecture for Computer Vision. CVPR.
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS. arXiv:1706.03762.
- Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture. ICML.
- 3Blue1Brown (2024). Neural Networks, capítulos 5 e 6. Vídeos (YouTube).